# PaperPhysics Final Plan (English)

Nov 6, 2026 · @Juntang Wang

## 1. Scope and hours

**PaperPhysics: draw a ball and a ramp on paper, take a photo, and it becomes a running simulation checked against the textbook formula. You can drag the ramp to change its angle, an AI teaching assistant follows your learning, and at the end you scan a QR code to take your animation home.**

- **Event:** Triangle AI Hackathon 2026, Duke University, Nov 6-8 (36 hours, teams of up to 4).
- **Track:** Computer Vision. The goal is to win the track (only track winners reach the Sunday final pitch), then compete for a top-3 overall prize.
- **Team:** 4 CS/Math undergraduates, working with Claude Code and Codex.
- **One-line difference:** the camera only *reads* the sketch, a physics engine *runs* it, SymPy *checks* it, and the AI assistant only speaks in numbers that have been verified.

**Scope principles**

- One problem only: a ball moving on a ramp. Only two drawn elements are required: a thick line (ground and ramp) and a circle (ball).
- Every phase ends with a version that can be demoed alone, so a problem in a later phase never breaks an earlier demo.
- Phases 1 and 2 are the floor and are never cut; everything else is cut in the order in section 11.

**Hours (estimates, not exact values)**

| Item | Person-hours |
|---|---|
| Must-do work for the four phases (including drag-to-adjust angle) | about 70 |
| The 7 optional items you chose | about 11.5 |
| Subtotal | about 81 |
| Integration buffer (about 10%) | about 7 |

| Item | Person-hours |
|---|---|
| **Total** | **about 88** |
| Team capacity (36 hours times 4 people, minus sleep, meals, workshops) | about 80 to 90 |

**An honest assessment: this is a tight plan.** The total of about 88 hours sits at the top of the available time. So we do not just hope to finish; we fixed the cut order and two decision points in advance (Saturday 12:00 and Saturday 5 PM). If we are behind at either, we cut optional items immediately, in order, instead of waiting until the end.

## 2. Four phases at a glance

**Four phases: about 70 person-hours of must-do work plus about 11.5 hours of chosen optional items; each phase has one clear demo moment.**

| Phase | Goal | Must-do output | Chosen optional items | Must-do hours | Optional hours | Demo moment |
|---|---|---|---|---|---|---|
| 1. Handwriting recognition to interactive dashboard | A photo becomes a runnable, verifiable simulation | Photo, recognize ground, ramp and ball, parameter panel, simulation, predicted vs simulated, formula | Sprite (ball), Gemini reads handwritten labels, manual four-corner fallback | about 31 | about 6 | "The ball I drew moved, and the formula checks out" |
| 2. Interaction and formula output | Turn the dashboard into a playable lab | Drag the ramp to change the angle, buttons for mass and gravity, rerun, side-by-side comparison, why-card | Friction button | about 11.5 | about 1 | "Drag the ramp and the prediction follows; double the mass and the time does not change" |

| Phase | Goal | Must-do output | Chosen optional items | Must-do hours | Optional hours | Demo moment |
|---|---|---|---|---|---|---|
| 3. Personalized physics TA | Remembers what you did, asks follow-ups, explains at your level | Session summary, answers grounded in verified numbers, follow-up questions, level choice, number check | Quiz me, TA explains recognition problems | about 12 | about 3 | "You haven't tried changing gravity yet, want to?" |
| 4. Export and share | Take one experiment home | Looping GIF, download page, QR code, deployment | Handwriting-style text card (handwriting tier A) | about 7 | about 1.5 | "Scan to take your animation" |
| Shared | Evaluation and storytelling | Test set and evaluation script, DevPost, pitch, backup video | none | about 8 | 0 | Accuracy numbers and a 3-minute script |
| **Total** | | | | **about 70** | **about 11.5** | |

# 3. Phase 1: from handwriting recognition to an interactive dashboard

**Goal: the user draws a ground line, a ramp and a ball on paper; after a photo they get a runnable simulation that matches the formula, shown in a dashboard where the parameters are visible.**

### Must-do (about 31 hours)

- [ ] **Capture (about 3 h):** find the paper, flatten it with a perspective warp, extract dark ink, split it into connected strokes.
- [ ] **Recognition (about 4 h):** recognize only two classes: thick line (ground or ramp) and circle (ball). Thick vs thin comes from stroke width.
- [ ] **Scene builder (about 3 h):** pixels to meters (default paper short side = 1 m); the ramp angle comes from the line's slope; mass and friction use defaults, with a "defaults used" notice in the UI.

- [ ] **Simulation (about 3.5 h):** build the ramp and ball in pymunk and record the time to reach the bottom. On Friday night, test "ball rolls" vs "ball slides" and pick whichever matches the formula.
- [ ] **Verification (about 3 h):** write the formulas in SymPy. Sliding block: `a = g(sin θ - μ cos θ)`, `t = sqrt(2L / a)`; rolling solid ball: `a = (5/7) g sin θ`. Output predicted value, simulated value and error; warn if the error is above 3 percent.
- [ ] **Dashboard (about 9.5 h):** upload page (works with a phone camera); photo with recognized outlines on the left, simulation canvas on the right; parameter panel (angle, mass, gravity, friction, showing current values); predicted vs simulated table; formula rendered with KaTeX; play and reset.
- [ ] **Evaluation and repo (about 4.5 h):** everyone draws 5 sketches; label file; one command prints accuracy; repo hygiene (`AGENTS.md`, `fixtures/`).

### Chosen optional items (about 6 hours)

- [ ] **Sprite, ball only (about 3 h, Vision lead):** cut the ball's ink into a transparent PNG and draw it in the simulation at the ball's position and rotation, so the rolling ball is the one the user drew. The ramp is still drawn as a line. A switch turns it off to fall back to the vector look. Accept: 80 percent of sketches give clean cutouts. **Precondition: do it only if Phase 1 is finished by Saturday 12:00.**
- [ ] **Gemini reads handwritten labels (about 2 h, Vision lead):** read labels such as "30°" and "2 kg" and fill them into the parameter panel; if unreadable, use defaults and say so in the UI; strict JSON output, no guessing.
- [ ] **Manual four-corner fallback (about 1 h, Vision lead 0.5 plus Frontend lead 0.5):** if paper detection fails, let the user tap the four corners on the photo. This is insurance and is cut last.

### Done when

- 3 prepared sketches and 1 live sketch all run with no manual help.
- Predicted and simulated values differ by 3 percent or less.
- On 25 test sketches, object recognition accuracy is at least 85 percent (the hard gate at Saturday 12:00 is 70 percent).

### Fallbacks

- Recognition is unstable: require a thick black marker, only lines and circles, and a fixed camera.
- Still unstable: go semi-automatic, with the user tapping to confirm the three objects.
- Rolling does not match the formula: demo with sliding and say so.

# 4. Phase 2: interaction, drag-to-adjust angle and formula output

**Goal: turn the dashboard into a playable lab: change one condition, rerun, compare, and explain why with a formula.** The angle is adjusted by dragging; mass and gravity use buttons; nothing depends on an LLM understanding free text.

**Must-do (about 11.5 hours)**

- [ ] **Three changes (about 4 h):** angle (drag the ramp, plus 15, 30 and 45 degree preset buttons), mass (a double button), gravity (Earth, Moon, Mars buttons). The engine reruns; the UI provides the controls.
- [ ] **Drag to adjust the angle (about 2.5 h):** drag the ramp with your hand and watch the prediction and the ball's motion change together.
  - The ramp's lower end is fixed and its upper end has a draggable handle; mouse and finger both work (Pointer events, so phones too); range 5 to 75 degrees, with a live angle readout while dragging and optional snapping to 5 degrees.
  - While dragging, "predicted time" and "predicted final speed" update live: the backend exposes the SymPy formula as a fast function (formula only, no simulation, millisecond latency); about 150 ms after release, the full simulation reruns and plays.
  - The originally drawn angle stays as a faint dashed line for comparison; a "reset angle" button and a number box are provided as fallbacks.
  - If the angle is too shallow for the ball to slide (friction coefficient greater than tan θ), show "the ball will not slide at this angle" and explain why.
- [ ] **Side-by-side comparison (about 1.5 h):** Run 1 and Run 2 play side by side with a comparison table (time, final speed).
- [ ] **Why-card (about 3.5 h):** show the LaTeX formula, the symbol that matters or cancels, and one plain sentence. SymPy decides what cancels (for example, the derivative of time with respect to mass is zero).

**Examples (a block sliding on an incline)**

| Change | What happens | Why-card |
|---|---|---|
| Double the mass | Time to the bottom does not change (the surprise) | `a = g(sin θ - μ cos θ)` has no `m`: mass cancels |
| Steeper ramp | Faster | `sin θ` grows with the angle, so `a` grows |

| Change | What happens | Why-card |
|---|---|---|
| On the Moon (g = 1.62) | About 2.46 times slower | `t ∝ 1/sqrt(g)`, sqrt(9.81 / 1.62) = 2.46 |

**Chosen optional item (about 1 hour)**

- [ ] **Friction button (about 1 h, Physics lead 0.5 plus Frontend lead 0.5):** add a friction-coefficient button. Example: when μ ≥ tan θ the ball stops, and the why-card shows that `μ cos θ` is subtracted from `sin θ`.

**Done when**

- The "double the mass" example works live every time; the Moon result matches 2.46 times.
- For every supported change, the symbolic prediction matches the rerun within 3 percent.
- Dragging: it works on a phone and a laptop; the prediction updates at least 10 times per second without lag; a new simulation result appears within 3 seconds of release.

**Fallback:** if dragging misbehaves, use the number box and the preset buttons; if that still fails, keep only the mass and angle changes.

## 5. Phase 3: personalized physics TA

**Goal: a teaching assistant that remembers what you did, asks follow-ups, explains at your level, and speaks only in numbers computed by the physics engine.**

**What "personalized" means here (two cheap, doable things)**

1. **Explain by level.** On entry the user taps their level (new to physics / know a little / fairly confident). The same conclusion is explained differently: everyday analogies and no formulas for beginners; direct formula derivation for confident users.
2. **Follow up on what you tried.** The TA tracks a concept list (mass independence, effect of angle, effect of gravity, effect of friction), sees which ones you have tried, and recommends the ones you have not.

**Learner profile (a small JSON kept in the session)**

```
{
  "level": "beginner",
  "concepts_seen": { "mass_independent": true, "angle_effect": true,
"gravity_effect": false, "friction_effect": false },
  "runs": ["r1", "r2", "r3"]
}
```

**Must-do (about 12 hours)**

- [ ] **Session log (about 1.5 h):** log every upload, run, change, post-drag rerun and question as an event. An in-memory list is enough; no database.
- [ ] **Context pack and prompt (about 2 h):** a compact context: the current scene, verified numbers for each run, the changes made, the learner profile. Prompt rules: answer only from the context, cite "Run N", write formulas in LaTeX, and say so when out of scope.
- [ ] **Number check (about 1 h):** a script extracts the numbers in the answer and compares them with the context pack; flag mismatches (the must-do version flags only and does not regenerate).
- [ ] **Summary (about 1.5 h):** a "summarize my session" button listing what you drew, the experiments, each conclusion and the formulas used. If the LLM fails, build it from a template directly from the log.
- [ ] **Follow-ups and personalized explanations (about 2.5 h):** after each answer, offer 2 follow-up buttons chosen from the concepts not yet tried; the level choice changes the explanation; concepts already explained are not repeated.
- [ ] **TA drawer UI (about 3 h):** a side drawer with a chat box, level choice, summarize button and follow-up buttons; Markdown and KaTeX; each answer shows "based on Run 1, Run 2".
- [ ] **Formula table (about 0.5 h):** the Physics lead writes a small table of which numbers and formulas each setup can report, for the TA's context.

**Chosen optional items (about 3 hours)**

- [ ] **Quiz me (about 2 h, LLM lead 1.5 plus Frontend lead 0.5):** the TA proposes a what-if (for example "how much faster at 45°?"), the user types a prediction first, then the real engine reruns and compares, giving "your prediction was off by x percent". All numbers come from the engine.
- [ ] **TA explains recognition problems (about 1 h, LLM lead):** when defaults are used or recognition confidence is low, the TA says what was read, what was assumed (for example mass 1 kg) and how to redraw.

**Done when**

- On 20 test questions, no answer contains an invented number.
- In 10 test sessions, the summary lists every change that was made.
- First token within 2 seconds, full answer within 6 seconds.
- The same question gets clearly different explanations for "new to physics" and "fairly confident".
- Quiz me: the prediction-vs-actual comparison always comes from the engine.

**Fallbacks**

- The summary uses a template (no LLM call).
- Follow-ups use fixed templates chosen from the concepts not yet tried.
- Drop the level choice and keep only the follow-ups.

## 6. Phase 4: export, QR code and handwriting-style card

**Goal: take an experiment home: generate a looping GIF, download it by scanning a QR code, and render the card's text in a handwriting-style font.**

**Must-do (about 7 hours)**

- [ ] **Replay recording and GIF (about 3 h, Vision lead):** replay a run on a canvas and record a 5 to 8 second looping GIF: photo on the left, simulation on the right, one line of predicted and simulated values below. GIF is chosen because it plays on every phone.
- [ ] **Share card layout (about 0.5 h, Frontend lead):** add a formula line and the project name.
- [ ] **Download page and QR code (about 1.5 h, Vision lead):** the backend stores the file and creates a download link; the frontend generates the QR code with a small library; scanning on a phone opens the download page.
- [ ] **Deployment (about 2 h, Vision lead 1 plus Frontend lead 1):** frontend on Vercel, backend on Cloud Run (using the $100 credits), or a temporary tunnel. Test the scan three times on the day.

**Chosen optional item (about 1.5 hours)**

- [ ] **Handwriting tier A: handwriting-style text card (about 1.5 h, Vision lead):** render the question and answer in a handwriting-style font on a paper background and put it in the export card. Note: this is a "handwriting style", not the user's own handwriting; say

so honestly in the pitch.

**Done when**

- Click to GIF ready in under 10 seconds.
- From scanning to downloading on your own phone, with no help, works 3 times out of 3.

**Fallbacks**

- Venue network does not work: point the QR code at the laptop's address on the same phone hotspot, or at a file uploaded in advance.
- GIF generation fails: export a still image with the numbers.
- QR code unusable: keep only a "Download GIF" button.

**Why only handwriting tier A:** tier B (reusing the user's own glyphs) needs the user's sketch to contain handwritten digits that can be cut out cleanly, which is poor value, and you chose to drop it; tier C (training a handwriting-synthesis model) is beyond 36 hours. I have not verified the effect or effort of those two tiers.

## 7. Confirmed scope: kept and dropped

**From the optional list you kept 7 items and dropped 9; drag-to-adjust angle was promoted to must-do.** This is the final scope.

**Kept (about 11.5 hours in total)**

| Phase | Optional item | Hours | Owner | Precondition and notes |
|---|---|---|---|---|
| 1 | Sprite, ball only | 3 | Vision | Only if Phase 1 is done by Saturday 12:00 |
| 1 | Gemini reads handwritten labels | 2 | Vision | If unreadable, use defaults; test accuracy on 10 labels first |
| 1 | Manual four-corner fallback | 1 | Vision plus Frontend | Insurance; cut last |
| 2 | Friction button | 1 | Physics plus Frontend | None |

| Phase | Optional item | Hours | Owner | Precondition and notes |
|---|---|---|---|---|
| 3 | Quiz me (predict first, then run) | 2 | LLM plus Frontend | Numbers must come from the engine |
| 3 | TA explains recognition problems | 1 | LLM | None |
| 4 | Handwriting tier A: handwriting-style font | 1.5 | Vision | In the pitch say "handwriting style", not "your handwriting" |

**Dropped (not doing)**

| Phase | Dropped optional item | Hours | Why dropped |
|---|---|---|---|
| 1 | Pendulum | 3 | Medium value, little connection to the main line |
| 1 | Initial-velocity arrow | 2 | Low value |
| 2 | Free-text what-if (LLM parsing) | 3 | Drag and buttons are enough, and parsing can fail |
| 3 | Hint and explain mode switch | 1 | The level choice and follow-ups are enough |
| 3 | Streaming output | 1 | Low value |
| 4 | Handwriting tier B: reuse the user's own glyphs | 4.5 | High risk, and probably no source material |
| 4 | Handwriting tier C: train a handwriting-synthesis model | n/a | Beyond 36 hours |
| 4 | Video format export | 1.5 | GIF is enough |
| 4 | Gallery wall page | 1.5 | Low value |

**If there is slack before Saturday 5 PM, add back in this order (cheapest and most worthwhile first):** hint and explain mode switch, streaming output, pendulum, free-text what-if. If there is no slack, add nothing.

# 8. Team roles and tasks

**Each person has one main job per phase, and later phases are picked up by whoever is free earlier.** Names are blank; fill in tonight. Hours are estimates.

| Role | Phase 1 | Phase 2 | Phase 3 | Phase 4 and other | Chosen optional items | Total hours |
|---|---|---|---|---|---|---|
| 1. Vision lead (name) | Capture and two-class recognition (7) | Help the Physics lead test scenes | Session log (1.5) | GIF, download page, QR code, deployment help (5.5) | Sprite 3, Gemini label reading 2, four-corner algorithm 0.5, handwriting-style card 1.5 | about 21 |
| 2. Physics and math lead (name) | Scene builder, pymunk, SymPy verification (10) | Change engine, live-prediction endpoint for dragging, why-card data (5.5) | Formula table (0.5) | DevPost technical section and content for pitch slides 3 and 4 (2) | Friction button 0.5 | about 18.5 |
| 3. Frontend lead (name) | Whole dashboard (9.5) | Drag handle, buttons, side-by-side, why-card UI (5.5) | TA drawer (3) | Share card layout, deployment (1.5) | Four-corner UI 0.5, friction button 0.5, quiz UI 0.5 | about 21 |
| 4. LLM, evaluation and pitch lead (name) | Repo hygiene, test set, evaluation script (4.5) | Evaluation adaptation (0.5) | TA brain: context pack, summary, follow-ups, personalization, number check (7) | Pitch, DevPost, backup video (6) | Quiz me 1.5, TA explains recognition problems 1 | about 20.5 |
| **Total** | | | | | | **about 81** |

### Why it is split this way

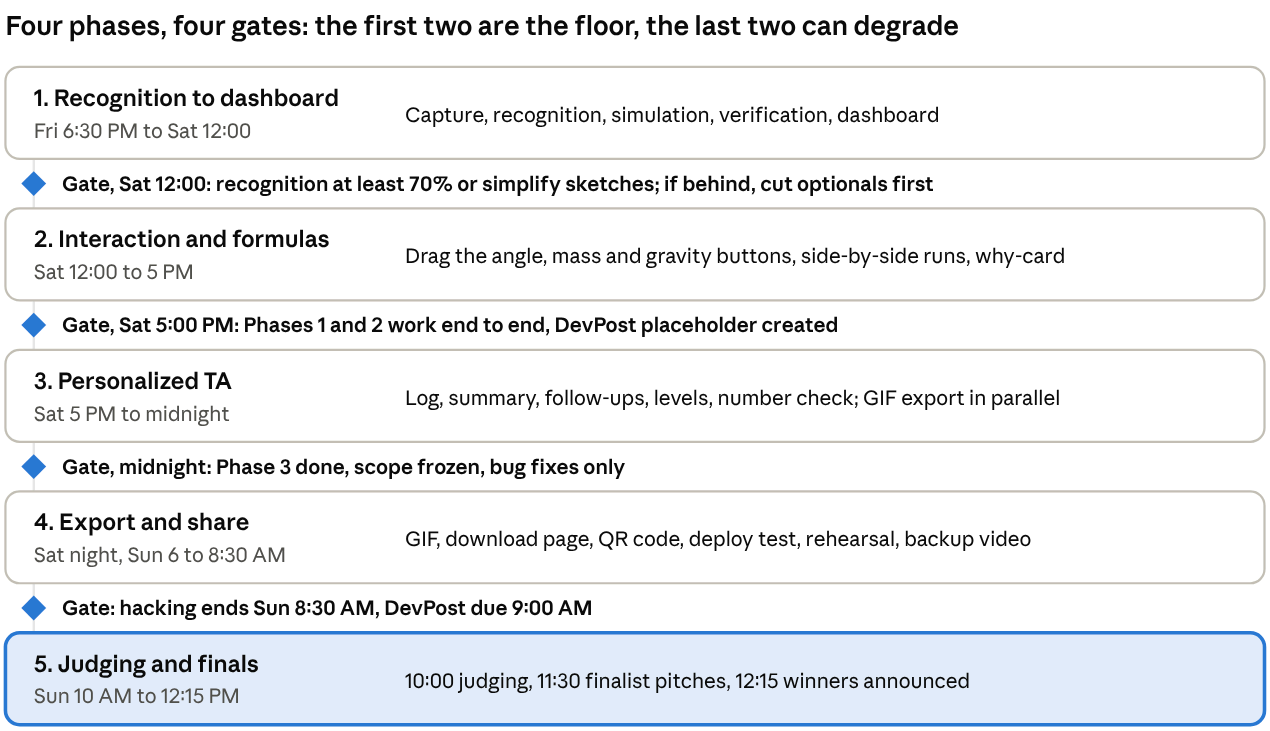
- The Vision lead is free once recognition is stable in Phase 1, so they take over export, the QR code and the session log.
- The LLM lead is light early on, so they own the whole TA brain in Phase 3: few interfaces, little communication.
- The Physics lead owns the most core engine and is relatively free on Saturday night, so they share the DevPost technical section and two pitch slides.

- Sprite rendering is done by the Vision lead (cutout and drawing together) to lighten the Frontend lead.

**Shared responsibilities**

- Everyone draws 5 test sketches on Friday night (different pens and handwriting).
- Agree the JSON contracts between modules in the first hour, then split up.
- Everyone reviews at least one other person's pull request; post a one-line status every 2 to 3 hours (done, next, blocked).
- Ask for help after 45 minutes stuck; decisions that block someone are made within 10 minutes.

# 9. Timeline




four-phase plan - four gates

The schedule lists kickoff at 5:30 PM Friday but no hacking start time, and says times can still shift, so treat these hours as a plan. Sleep in shifts: about 5 hours each, and no new features after midnight Saturday. **The Phase 4 GIF export is built in parallel by the Vision lead on Saturday night and does not wait for Phase 3 to finish**; Sunday morning is only for the QR code, deployment and rehearsal. Where the chosen optional items land: sprite and

Gemini label reading right after Phase 1, the friction button in Phase 2, Quiz me and the recognition-problem explanations in Phase 3, handwriting tier A on Saturday night together with the GIF.

## 10. 3-minute demo

**The demo follows the four phases: draw, snap, watch it move and be verified, play, ask the TA, take it home.**

1. **Hook (10 s):** "Every physics student has drawn a ramp and a ball. What if the drawing could run?"
2. **A judge draws (25 s):** hand a judge a marker and paper; they draw a ground line, a ramp and a ball, and may write "30°" next to the ramp.
3. **Snap (10 s):** take the photo and upload it.
4. **Phase 1, dashboard (45 s):** recognized outlines show on the photo; the read "30°" fills the parameter panel; *the ball the judge drew* rolls down the ramp (sprite); "predicted 1.42 s, simulated 1.41 s" appears with the formula.
5. **Phase 2, play (40 s):** drag one end of the ramp to change the angle (the predicted time updates live, and the ball rolls again on release), then tap *double the mass*: the time does not change and the why-card shows that mass cancels; tap *Moon*: about 2.5 times slower.
6. **Phase 3, ask the TA (25 s):** tap "summarize my session"; the TA lists the experiments, explains at the chosen level, and says "you haven't tried changing friction yet"; if time allows, tap "quiz me" and let the judge guess the time at 45°.
7. **Phase 4, take it home (15 s):** generate the GIF (handwriting-style text card), show the QR code and invite the judges to scan it.
8. **Close (10 s):** who benefits and what comes next.

### Demo rules

- Prepare 3 tested sketches plus the judge's live one; give a thick black marker; fix the camera and lighting.
- If handwritten labels cannot be read, fall back to defaults; do not depend on them live.
- Phases 3 and 4 come last, so if something fails live, the first two thirds of the demo still stand.
- Pre-generate one GIF and a QR poster in case live export or the network misbehaves; record a backup video; keep a phone hotspot ready.

# 11. Risks, cut order and gates

**The biggest risk is that real hand-drawn sketches are messier than test images; the second is overrunning hours (about 88 in total, at the top of capacity).**

| Risk | Impact | Mitigation | Owner |
|---|---|---|---|
| Recognition unstable on wobbly drawings | The whole chain fails | Thick marker, fixed camera; test on Friday night; if it fails, go to tap-to-confirm of the three objects | Vision lead |
| pymunk rolling does not match the formula | Large verification error | Test rolling and sliding on Friday night; choose the one that matches | Physics lead |
| Dragging is not smooth on a phone, or the prediction endpoint is slow | The "play" step falls over | Test on a phone on Saturday morning; if slow, fall back to the number box and preset buttons | Frontend lead |
| The TA invents numbers | Credibility lost | Answer only from the context pack; the number check flags mismatches; template summary as a fallback | LLM lead |
| Gemini misreads handwritten labels | Wrong parameters | Show the read value in the panel and let it be edited; if unreadable, use defaults | Vision lead |
| The QR code and download do not work on venue network | The "take it home" step fails | Deploy early; hotspot; poster QR; still image and file download as fallbacks | Vision lead |
| Hours overrun | Later phases do not finish | Follow the cut order below; do not add features on the fly | Everyone |
| AI tool limits, merge conflicts, fatigue | Slower progress | Stagger use; one module per person; sleep in shifts; no new features after Saturday midnight | Everyone |

**Cut order (from first to last; if we overrun, drop in this order)**

1. Handwriting tier A: handwriting-style card (1.5 h).
2. TA explains recognition problems (1 h).
3. Quiz me (2 h).
4. Friction button (1 h).
5. Gemini reads handwritten labels (2 h).
6. Sprite (3 h).
7. Must-do: the Phase 4 QR code (keep the Download GIF button).
8. Must-do: the Phase 3 level choice (keep the summary and follow-ups).
9. **Never cut: the manual four-corner fallback (insurance), Phase 1, and Phase 2 (including drag-to-adjust angle).**

**Gates**

- **Saturday 12:00:** object recognition at least 70 percent, otherwise simplify the sketch language; also check progress and, if behind, cut items 1 to 3 first.
- **Saturday 5 PM:** Phases 1 and 2 run end to end and the DevPost placeholder exists; if behind, keep cutting items 4 to 6.
- **Saturday midnight:** Phase 3 done; freeze scope.
- **Sunday 8:30 and 9:00 AM:** hacking ends and DevPost is submitted.

## 12. What to do tonight

**Tonight's goal: everyone claims a role, the contracts are agreed, and one hand-drawn sketch is shown to become a simulation checked by a formula.**

- [ ] The four of us claim the roles (section 8).
- [ ] Create the repo, `AGENTS.md` (plus a `CLAUDE.md` pointing to it), `fixtures/` and the `.env` setup; agree the JSON contracts between modules in the first hour.
- [ ] Everyone draws 5 test sketches (at least a ramp and a ball; on some, write labels such as "30°" and "2 kg"), photographed in the same spot.
- [ ] Vision lead: paper detection plus line and circle recognition, and report the first accuracy number; also try Gemini on 10 handwritten labels to measure how accurately it reads them.

- [ ] Physics lead: build the ramp and ball in pymunk, compute the formulas in SymPy, test whether "rolling" or "sliding" matches the formula, and verify that the derivative of time with respect to mass is zero.
- [ ] Frontend lead: scaffold, a phone upload page and a split view with fake data; try a draggable handle on a phone to feel how it behaves.
- [ ] LLM lead: test-set label sheet and an evaluation-script skeleton; ask the organizers three things (registered teams per track, judging criteria, whether the Entrinsik Challenge can be combined with a track); draft the TA system prompt.
- [ ] Vision and Frontend leads: decide how to deploy (the QR code needs a public address: Vercel plus Cloud Run, or a tunnel).
- [ ] Attend the workshops: Intro to Computer Vision (7:45 PM, Wilkinson 132) and Getting Started with Vibe Coding (7:00 PM, Wilkinson 126).

**Note:** four things I have not verified and that could change the plan, so test them tonight: how drag feels on a phone, the speed of the live-prediction endpoint, how accurately Gemini reads handwritten labels, and how accurately pymunk rolling matches the formula.